

ASSIGNMENT 3
Software Development and Integration

Аналитический отчёт

**Организация разработки, выбор технологий, системы контроля версий
и CI/CD на примере программного модуля раннего выявления
профессионального выгорания по речи**

Выполнил: Zhalgas Karsenbay
Репозиторий: <https://github.com/jksenby/burnout-employee-predictor>

Введение

Тема магистерской диссертации автора — раннее выявление профессионального выгорания сотрудников по речевым признакам. Программная часть исследования реализована как веб-система *Burnout Employee Predictor*. Сервер на FastAPI принимает аудиозаписи, извлекает из них акустические, эмоциональные и лингвистические признаки, оценивает риск выгорания классификатором и сравнивает каждую новую запись с личной нормой человека, накопленной за первые сессии. Клиент на React проводит пользователя через протокол наблюдения: опросник Маслач (MBI) раз в 60 дней, еженедельные записи речи в двух режимах (свободное интервью и чтение) и итоговый отчёт.

Цель работы — изучить организацию процесса разработки программного обеспечения, обосновать выбор технологий, освоить системы контроля версий и практики непрерывной интеграции и доставки (CI/CD) и применить их к реальному научному программному модулю. Первые пять разделов отчёта посвящены теории, шестой описывает практическую часть: оформление репозитория на GitHub, автоматическое тестирование и конвейер GitHub Actions. В заключении подведены итоги и перечислены трудности, возникшие по ходу работы.

1. Этапы и организация процесса разработки ПО

Жизненный цикл программного обеспечения (Software Development Life Cycle, SDLC) описывает путь продукта от идеи до вывода из эксплуатации. Стандарт ISO/IEC/IEEE 12207 выделяет множество процессов, но на практике их обычно сводят к шести этапам.

- **Анализ требований.** Определяются цели, пользователи, функциональные и нефункциональные требования (производительность, безопасность, приватность данных). Результат — спецификация или бэклог пользовательских историй.
- **Проектирование.** Выбирается архитектура, модель данных, интерфейсы между модулями и технологический стек.
- **Реализация.** Код пишется небольшими проверяемыми порциями, проходит ревью и фиксируется в системе контроля версий.
- **Тестирование.** Модульные, интеграционные и сквозные тесты подтверждают, что система делает то, что требуется, и что новые изменения не ломают старое поведение.
- **Развёртывание (поставка).** Собранный артефакт передаётся в эксплуатацию.
- **Сопровождение.** Исправляются ошибки, обновляются зависимости, система развивается по обратной связи.

Порядок и длительность этапов задаёт модель процесса. В **каскадной** модели (Waterfall) этапы идут строго последовательно, поэтому она подходит для проектов со стабильными требованиями, но плохо переносит изменения: ошибка в требованиях

обнаруживается только при тестировании. **V-модель** ставит каждому этапу проектирования в соответствие уровень тестирования. **Итеративные и инкрементные** модели, в том числе спиральная модель Боэма, проходят все этапы многократно и на каждой итерации наращивают функциональность. **Гибкие** подходы (Agile) — Scrum с фиксированными спринтами и ролями и Kanban с непрерывным потоком задач и лимитом незавершённой работы — ставят во главу угла работающий продукт, короткую обратную связь и готовность к изменениям. Подход **DevOps** добавляет к ним автоматизацию сборки, тестирования и поставки и общую ответственность разработки и эксплуатации за результат.

Научное ПО почти всегда развивается итеративно: требования уточняются вместе с гипотезами исследования. История репозитория этого проекта (36 коммитов с декабря 2025 года) это хорошо иллюстрирует. Сначала был создан минимальный конвейер «аудио → признаки → оценка». Затем добавились опросник MBI, интернационализация интерфейса, разделение речевого анализа на два режима и личная норма голоса. Расчёт jitter, shimmer и HNR был переписан на Praat после того, как проверка на синтетических сигналах показала несостоятельность прежних формул. Задачи велись по сути в стиле Kanban, где каждая итерация заканчивалась работающей версией.

2. Критерии выбора технологий и инструментов

Выбор стека — инженерное решение с долгосрочными последствиями: сменить язык или фреймворк в зрелом проекте дорого. Обычно учитывают следующие критерии.

- **Соответствие задаче:** есть ли готовые библиотеки для предметной области.
- **Зрелость и сообщество:** документация, частота релизов, число пользователей, ответы на типовые вопросы.
- **Производительность и масштабируемость** на ожидаемой нагрузке.
- **Порог входа и компетенции команды.**
- **Совместимость и интеграция** с остальными частями системы.
- **Лицензия:** допускает ли она академическое и коммерческое использование.
- **Воспроизводимость и переносимость:** насколько легко поднять окружение на другой машине или в CI. Для научного ПО это критично.
- **Безопасность и сопровождаемость:** обновления, известные уязвимости, долгосрочная поддержка.

В таблице 1 приведено обоснование выбора по каждой подсистеме проекта.

Таблица 1 — Обоснование выбора технологий

Подсистема	Выбор	Альтернативы	Обоснование
Язык сервера	Python 3.11+	R, Julia, C++	Стандарт де-факто для ML и обработки речи: PyTorch, transformers, librosa, scikit-learn, Praat-биндинги доступны из одного языка
Веб-API	FastAPI	Flask, Django	Асинхронность (запись обрабатывается десятки секунд), валидация через Pydantic, автоматическая документация OpenAPI, встроенный TestClient
БД / ORM	SQLite + SQLAlchemy 2	PostgreSQL	Достаточно для пилотного исследования, без отдельного сервера; ORM позволяет перейти на PostgreSQL сменой строки подключения
Акустика	Praat (parselmouth), librosa	openSMILE, самописные формулы	Praat — эталон фонетических измерений (jitter, shimmer, HNR) в научной литературе; librosa — MFCC и спектральные признаки
Нейросетевые признаки	HuBERT, WavLM, wav2vec2 (HuggingFace)	собственное обучение	Предобученные самоконтролируемые модели дают устойчивые представления речи без большого размеченного корпуса
Распознавание речи	faster-whisper (CTranslate2)	openai-whisper, Vosk	В 2–4 раза быстрее оригинала при той же точности, поддерживает казахский и русский, работает на CPU
Классификатор	Gradient Boosting (scikit-learn)	нейросеть, SVM	Работает на малых выборках, интерпретируется через важность признаков
Клиент	React 19, react-i18next, Recharts	Vue, Angular	Большая экосистема, запись звука через MediaRecorder, графики динамики, RU/KZ/EN
Тестирование	pytest, pytest-cov, Jest	unittest	Фикстуры, параметризация, отчёты покрытия; Jest входит в react-scripts
Линтер	ruff	flake8, pylint	Работает на порядки быстрее и заменяет несколько инструментов
CI/CD	GitHub Actions	GitLab CI, Jenkins	Встроен в платформу хостинга, бесплатен для публичных репозиторий, не нужен свой сервер

3. Системы контроля версий: обзор и сравнение

Система контроля версий (VCS) хранит историю изменений файлов. С ней можно вернуться к любому состоянию, понять, кто и зачем изменил строку, и вести параллельную работу в ветках. **Централизованные** системы (CVS, Subversion) держат историю на одном сервере, и без сети почти ничего нельзя сделать. В **распределённых** системах (Git, Mercurial) у каждого разработчика полная копия репозитория: коммиты, ветвление и просмотр истории работают локально, а синхронизация идёт через push/pull.

Таблица 2 — Сравнение систем контроля версий

Критерий	Git	Mercurial	Subversion (SVN)
Архитектура	распределённая	распределённая	централизованная
Ветвление	очень дешёвое, ветка — указатель на коммит	дешёвое (bookmarks, named branches)	копия каталога на сервере
Работа офлайн	полностью	полностью	только чтение рабочей копии
Целостность	SHA-1/SHA-256 для каждого объекта	SHA-1	номера ревизий
Большие бинарные файлы	через Git LFS	через расширение largefiles	хорошо, есть блокировки файлов
Распространённость	доминирует (>90% проектов)	нишевая	легаси, геймдев

Git стал стандартом благодаря скорости, модели ветвления и экосистеме платформ хостинга. Самые известные из них — GitHub, GitLab и Bitbucket. Кроме хранения кода, они дают задачи (issues), запросы на слияние (pull/merge requests) с ревью, вики, встроенный CI/CD и управление релизами.

Таблица 3 — Сравнение платформ хостинга Git

Возможность	GitHub	GitLab	Bitbucket
CI/CD	GitHub Actions	GitLab CI (встроен с 2015 г.)	Bitbucket Pipelines
Самостоятельный хостинг	Enterprise Server	Community Edition (open source)	Data Center
Сообщество	крупнейшее, основная площадка open source и науки	сильное в DevOps и on-premise	интеграция с Jira
Научные интеграции	Zenodo (DOI для релизов), GitHub Pages, Codespaces	Pages, встроенный реестр контейнеров	ограниченные

Над Git строятся соглашения о ветвлении. **Git Flow** разделяет ветки develop, feature, release и hotfix и подходит для продуктов с плановыми релизами. **GitHub Flow** проще: от основной ветки создаётся короткоживущая ветка под задачу, открывается pull

request, CI проверяет изменения, после ревью ветка вливается. **Trunk-based development** сводит ветки к минимуму и требует сильного CI. Для одного исследователя с научным руководителем в роли ревьюера оптимален GitHub Flow, и он использован в практической части. Хорошая практика — атомарные коммиты с осмысленными сообщениями, файл .gitignore, исключающий артефакты и данные, и защита основной ветки правилом «слияние только после зелёного CI».

4. Непрерывная интеграция и доставка (CI/CD)

Непрерывная интеграция (Continuous Integration) — практика, при которой разработчики часто, не реже раза в день, вливают изменения в общую ветку, а каждое изменение автоматически собирается и проверяется тестами. Так ошибки находятся через минуты после появления, пока контекст ещё свеж, а не при ручном тестировании перед релизом. **Непрерывная доставка** (Continuous Delivery) продолжает CI: любая версия, прошедшая конвейер, собрана в артефакт и готова к выпуску одним действием. **Непрерывное развёртывание** (Continuous Deployment) убирает и это действие: каждое изменение, прошедшее проверки, автоматически уходит в эксплуатацию.

Конвейер (pipeline) состоит из стадий, и каждая следующая дорожке предыдущей. Типичный порядок такой: статический анализ и линтинг, сборка, модульные тесты, интеграционные тесты, анализ покрытия и безопасности, упаковка артефакта, публикация или развёртывание. Ключевые практики:

- **Быстрая обратная связь:** дешёвые проверки идут первыми (fail fast), зависимости кэшируются.
- **Воспроизводимая сборка:** зафиксированные версии зависимостей и чистое окружение на каждый запуск.
- **Сборка один раз:** на всех стадиях используется один и тот же артефакт.
- **Матрица окружений:** проверка на нескольких версиях языка и ОС.
- **Конвейер как код:** конфигурация хранится в репозитории рядом с кодом и проходит ревью.
- **Секреты вне кода:** ключи хранятся в защищённом хранилище платформы.
- **Защищённая ветка:** слияние запрещено, пока проверки не прошли.

Таблица 4 — Сравнение инструментов CI/CD

Инструмент	Модель	Сильные стороны	Ограничения
GitHub Actions	SaaS, YAML в <code>.github/workflows</code>	Маркетплейс готовых действий, матрицы, бесплатно для публичных репозиторий	Привязка к GitHub, лимиты минут для частных
GitLab CI	SaaS или self-hosted, <code>.gitlab-ci.yml</code>	Глубокая интеграция с GitLab, свои раннеры, реестры	Привязка к GitLab
Jenkins	self-hosted сервер, <code>Jenkinsfile</code>	Максимальная гибкость, тысячи плагинов	Сервер нужно администрировать и обновлять

5. Особенности разработки научного программного обеспечения

Научное ПО отличается от промышленного прежде всего целью. Результат здесь — не продукт, а достоверный вывод. Ошибка в коде может не уронить программу, а молча исказить число в таблице статьи. Отсюда особые требования.

Требования

- **Воспроизводимость:** другой исследователь должен получить те же результаты на тех же данных. Для этого нужны фиксированные версии библиотек, зерна генераторов случайных чисел и сохранённые параметры экспериментов.
- **Корректность:** проверка на эталонных входах с известным ответом, например на синтетическом сигнале с заданной частотой основного тона.
- **Прослеживаемость:** по любому результату можно установить версию кода, модели и данных, которыми он получен.
- **Этичность и приватность данных:** аудиозаписи голоса и ответы на опросники — персональные данные, они не должны попадать в публичный репозиторий.

Окружения и библиотеки

Окружение изолируют виртуальными средами (venv, conda) или контейнерами (Docker), а версии фиксируют в `requirements.txt`, `environment.yml` или lock-файлах. Основа научного стека Python — NumPy, SciPy, pandas, scikit-learn и Matplotlib, а также предметные библиотеки (здесь librosa, Praat/parselmouth, PyTorch и transformers). Тяжёлые зависимости, такие как PyTorch и веса нейросетей размером в несколько гигабайт, затрудняют CI. Поэтому полезно отделять вычислительное ядро от инфраструктуры, чтобы ядро можно было тестировать без них.

Лучшие практики

Сообщество выработало набор рекомендаций: Wilson et al., «Best Practices for Scientific Computing» (2014) и «Good Enough Practices in Scientific Computing» (2017), а также принципы FAIR для исследовательского ПО. Главные из них:

- контроль версий для кода, конфигураций и метаданных моделей, но не для сырых персональных данных;
- автоматические тесты с допусками для численных результатов (`pytest.approx`), тесты на синтетических данных с известным ответом и регрессионные тесты;
- явное версионирование схем данных: если меняется формула признака, старые и новые результаты нельзя смешивать;
- README с инструкцией запуска, лицензия (MIT, BSD, Apache-2.0) и при публикации архивирование релиза с DOI (GitHub → Zenodo);
- документирование ограничений метода прямо в коде и в README.

В проекте эти практики уже частично применялись до начала работы. Версия схемы признаков `FEATURE_SCHEMA_VERSION` записывается и в обученную модель (`burnout_model.meta.json`), и в каждую запись в БД, а несовпадение версий явно сообщается при запуске. Генератор обучающих данных использует фиксированное зерно. Метод личной нормы подробно задокументирован в комментариях. В практической части к этому добавлены автоматические тесты и CI.

6. Практическая часть

6.1. Репозиторий и его оформление

Проект опубликован в публичном репозитории <https://github.com/jksenby/burnout-employee-predictor>. В рамках задания репозиторий дополнен следующими файлами:

- **README.md** — назначение системы, схема конвейера анализа, структура модулей, инструкции по установке, запуску, тестированию и выпуску релиза, ограничения метода, бейдж статуса CI;
- **LICENSE** — лицензия MIT;
- **.github/ISSUE_TEMPLATE/** — шаблоны задач для ошибок и предложений; в шаблон предложения встроен вопрос о влиянии на методику (схему признаков, личную норму, модель);
- **.github/pull_request_template.md** — чек-лист для pull request;
- **.github/dependabot.yml** — ежемесячные автоматические PR с обновлениями `pip`, `npm` и GitHub Actions;
- **.gitignore** — исключает локальную базу данных `users.db`, окружения и отчёты тестов.

Работа велась по GitHub Flow: изменения сделаны в ветке *assignment-3-ci* серией атомарных коммитов (тесты, исправления, CI, документация) и вливаются в master через pull request после прохождения CI.

6.2. Автоматическое тестирование

До начала работы в проекте не было ни одного работающего теста. Единственный frontend-тест был шаблоном Create React App и искал на странице текст «learn react», которого в приложении нет. Написаны тесты трёх уровней (таблица 5).

Таблица 5 — Наборы автоматических тестов

Набор	Что проверяется	Тестов
test_analysis.py	Личная норма: калибровка, стабильность на той же записи, направление «хуже/лучше», ограниченность оценки, пол масштаба при нулевом MAD, отбрасывание вырожденных записей и нечисловых значений	11
test_audio.py	Декодирование (ресэмплинг, стерео → моно), отказ на пустых, коротких, тихих и клипированных записях; Praat-признаки на синтетическом голосе с известной частотой F0; шум распознаётся как невокализованный	10
test_text_features.py	Лексические маркеры для английского, русского (флексивные формы) и казахского, хеджирование, диапазоны значений	7
test_model.py	Согласованность признаков с метаданными модели, проверка версии схемы, нейтральные значения без транскрипта, порядок оценок эвристики, контракт выхода обученной модели	10
test_auth_and_content.py	bcrypt и JWT, подсчёт субшкал MBI, ротация вопросов интервью	11
test_api.py	Сквозные HTTP-сценарии: регистрация, вход, MBI, анализ речи, коды ошибок, полный протокол наблюдения → отчёт и PDF	10
formatters.test.js	Форматирование значений в интерфейсе (Jest)	9

Главная сложность — нейросетевые модули. HuBERT, WavLM, wav2vec2 и Whisper требуют PyTorch и нескольких гигабайт весов, поэтому запускать их в CI на каждый коммит непрактично. Решение: в сквозных тестах эти четыре модуля подменяются заглушками через sys.modules, а БД — SQLite в памяти. Всё остальное работает по-настоящему: декодирование WAV, Praat, текстовые признаки, обученный классификатор, личная норма, SQLAlchemy и генерация PDF. Входной сигнал синтезируется с известными свойствами: гармонический тон 120–160 Гц, слоговая модуляция 4 Гц и паузы. Поэтому тесты проверяют не только то, что код не падает, но и правильность измерений. Покрытие вычислительного ядра составляет 92–95% (analysis.py — 93%, feature_extraction.py — 92%, model.py — 93%, text_features.py — 95%, audio_io.py — 95%), main.py с эндпоинтами — 66%.

6.3. Конвейер CI/CD

Конвейер реализован на GitHub Actions и состоит из двух файлов: [ci.yml](#) и [release.yml](#) (полные ссылки приведены в конце раздела). CI запускается на каждый push и pull request и включает три задания:

- **lint** — ruff ищет ошибки, которые проявятся только при выполнении: неопределённые имена, синтаксис, неиспользуемые импорты;
- **backend** (после lint) — pytest с отчётом покрытия на матрице Python 3.11 и 3.12; зависимости из облегчённого requirements-ci.txt без PyTorch кэшируются; отчёты coverage.xml и JUnit сохраняются как артефакты;
- **frontend** (параллельно) — npm ci, Jest и production-сборка с CI=true, при которой любое предупреждение ESLint считается ошибкой; собранный фронтенд сохраняется как артефакт.

```
jobs:
  lint:      ruff check .
  backend:   needs: lint
             matrix: python-version: ["3.11", "3.12"]
             pip install -r requirements-ci.txt
             pytest --cov --cov-report=xml --junitxml=pytest-report.xml
  frontend:  npm ci && npm test -- --watchAll=false && npm run build
```

Непрерывная доставка (release.yml) срабатывает на тег вида v1.0.0. Через workflow_call она переиспользует весь CI, и релиз не собирается, если тесты не прошли. Затем она публикует GitHub Release с архивом собранного фронтенда, обученной моделью, её метаданными и файлом контрольных сумм SHA-256. Такой релиз фиксирует точное состояние кода и модели, с которым получены результаты диссертации, а связка GitHub с Zenodo может присвоить ему DOI.

6.4. Ссылки

Репозиторий: <https://github.com/jksenby/burnout-employee-predictor>

Конфигурация CI:

<https://github.com/jksenby/burnout-employee-predictor/blob/master/.github/workflows/ci.yml>

Конфигурация CD (релизы): <https://github.com/jksenby/burnout-employee-predictor/blob/master/.github/workflows/release.yml>

Запуски GitHub Actions: <https://github.com/jksenby/burnout-employee-predictor/actions>

Заключение

В работе рассмотрены этапы и модели процесса разработки, критерии выбора технологий, системы контроля версий и платформы хостинга, практики CI/CD и особенности научного ПО. Программный модуль диссертации доведён до состояния воспроизводимого открытого проекта. У него есть README, лицензия и шаблоны задач, 68 автоматических тестов (59 на Python и 9 на JavaScript) и конвейер GitHub Actions с линтингом, тестами на двух версиях Python, сборкой клиента и автоматическим выпуском релизов.

Трудности и выводы.

- **Тяжёлые зависимости.** Полный requirements.txt включает PyTorch, transformers и Whisper. Решено разделением зависимостей (requirements-ci.txt) и заглушками нейросетевых модулей в тестах. Вывод на будущее: вычислительное ядро стоит с

самого начала отделять от тяжёлой инфраструктуры.

- **CI сразу нашёл реальную ошибку.** Линтер выявил в эндпоинте `/report/pdf` обращение к неопределённым переменным `mbi_count` и `speech_count`. Они пропали при рефакторинге расчёта допуска к отчёту. В результате генерация PDF падала с ошибкой 500 ровно в тот момент, когда пользователь завершал протокол наблюдения. Ошибка исправлена и закреплена регрессионным тестом.
- **Устаревшие и «тихие» проверки.** Шаблонный тест Create React App всегда падал, а три предупреждения ESLint (мёртвый код и неполные зависимости `useEffect`) блокировали сборку в режиме CI. Всё исправлено.
- **Персональные данные в репозитории.** Локальная БД `users.db` с учётными записями и результатами анализов была под контролем версий. Она исключена из индекса и добавлена в `.gitignore`, но остаётся в истории коммитов. Для полной очистки нужна перезапись истории (`git filter-repo`) и смена паролей тестовых учётных записей.
- **Ограничение метода.** Классификатор пока обучен на синтетических данных, поэтому его абсолютная оценка демонстрационная. Основным научным сигналом остаётся отклонение от личной нормы. Следующий шаг — сбор реального размеченного корпуса и переобучение, при котором новые признаки и модели будет страховать уже готовый CI.

Список источников

1. ISO/IEC/IEEE 12207:2017. Systems and software engineering — Software life cycle processes.
2. Chacon S., Straub B. Pro Git. 2nd ed. Apress, 2014. <https://git-scm.com/book>
3. Humble J., Farley D. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley, 2010.
4. Fowler M. Continuous Integration. 2006.
<https://martinfowler.com/articles/continuousIntegration.html>
5. Wilson G. et al. Best Practices for Scientific Computing // PLoS Biology. 2014. 12(1): e1001745.
6. Wilson G. et al. Good Enough Practices in Scientific Computing // PLoS Computational Biology. 2017. 13(6): e1005510.
7. Barker M. et al. Introducing the FAIR Principles for research software // Scientific Data. 2022. 9: 622.
8. GitHub Actions documentation. <https://docs.github.com/actions>
9. Boersma P., Weenink D. Praat: doing phonetics by computer. <https://www.praat.org>
10. Maslach C., Jackson S. E. The measurement of experienced burnout // Journal of Occupational Behaviour. 1981. 2(2): 99–113.